UNIVERSIDAD SIGLO 21

Licenciatura en Informática

Trabajo práctico AP4

Proyecto integrador final

Seminario de Práctica de Informática

SIGTAS

Sistema de Gestión de Turnos y Atención
Centro Médico Salud Integral

Alumno: Pablo Ruiz

Modalidad: individual

Fecha: noviembre de 2026

Entrega incremental basada en el TP3.
Organización ficticia utilizada con fines académicos.

1. Continuidad, problema y modelo de negocio

SIGTAS continúa el proyecto del TP3: centralizar la gestión de turnos del Centro Médico Salud Integral. El problema es la dispersión de información en planillas y mensajes, que dificulta conocer la disponibilidad, puede generar reservas superpuestas y debilita la trazabilidad de las cancelaciones.

Objetivo general. Desarrollar un prototipo operacional en Java y MySQL que consulte, registre y cancele turnos, con validaciones, persistencia y separación de responsabilidades. El alcance conserva la consola y los catálogos precargados del proyecto anterior.

Dimensión	Análisis y justificación
Usuarios y beneficiarios	Recepción opera el sistema; profesionales consultan una agenda consistente; pacientes se benefician de reservas coordinadas.
Proceso actual	Recepción verifica disponibilidad y registra cambios en fuentes dispersas. La conciliación requiere intervención manual.
Proceso propuesto	Una consulta centralizada informa los turnos; una reserva valida disponibilidad y registra auditoría; la cancelación conserva historial.
Valor y recursos	Reducir retrabajo y duplicaciones. Se requiere equipo de desarrollo, base de datos, capacitación y mantenimiento. No se dispone de costos medidos.
Indicadores propuestos	Tiempo medio para reservar, cantidad de superposiciones y proporción de cancelaciones con motivo. No se afirma una mejora cuantificada sin medición.

2. Proceso Unificado de Desarrollo

Inicio: se mantiene el problema, los actores y los casos de uso de consulta, alta y cancelación. **Elaboración:** se refina el contrato DAO, las reglas de agenda y el esquema relacional. **Construcción:** se implementan Java, JDBC, transacciones, arreglos, ArrayList y archivos. **Transición:** se prepara instalación, pruebas, documentación y demostración.

La iteración prioriza riesgos concretos: persistencia inconsistente, reservas concurrentes y divergencias entre documentación y código. Los casos de uso guían las pruebas. La arquitectura por capas permite ampliar la interfaz y los catálogos sin concentrar cambios en una sola clase.

Continuidad: no se adjuntaron observaciones docentes específicas. Se incorporan mejoras técnicas identificadas al comparar el TP3 con el código público: validación completa de intervalos, control del día de agenda, auditoría transaccional y pruebas automatizadas. No se atribuyen estas mejoras a comentarios docentes no disponibles.

3. Arquitectura y patrón de diseño

Se selecciona **DAO (Data Access Object)** porque el problema requiere encapsular el acceso a MySQL y mantener la lógica de negocio separada del SQL. Oracle (s. f.-a) describe este patrón como una forma de abstraer el mecanismo de acceso a datos. La elección es coherente con las capas ya definidas en el TP3.

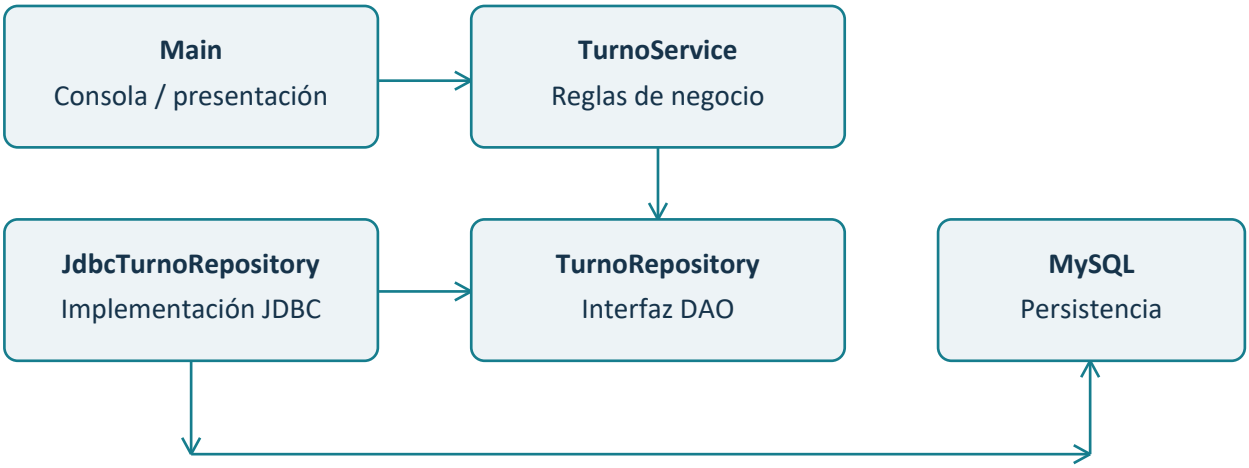


Figura 1. Dependencias principales de SIGTAS. Main utiliza el servicio para las operaciones de negocio y el repositorio para consultas. TurnoService depende de la interfaz; JdbcTurnoRepository implementa ese contrato y accede a MySQL. La flecha entre implementación e interfaz representa implementación, no una llamada SQL.

Componente	Responsabilidad
ui.Main	Captura datos, invoca operaciones y presenta resultados o errores.
service.TurnoService	Valida identificadores, fechas, intervalos y motivo de cancelación. Recibe TurnoRepository por constructor.
repository.TurnoRepository	Declara guardar, cancelar, buscarPorId y agendaDiaria.
repository.JdbcTurnoRepository	Implementa consultas parametrizadas, validación persistente, bloqueo, transacción y mapeo de resultados.
config.DatabaseConnection	Crea conexiones con configuración externa. No conserva una conexión global compartida.

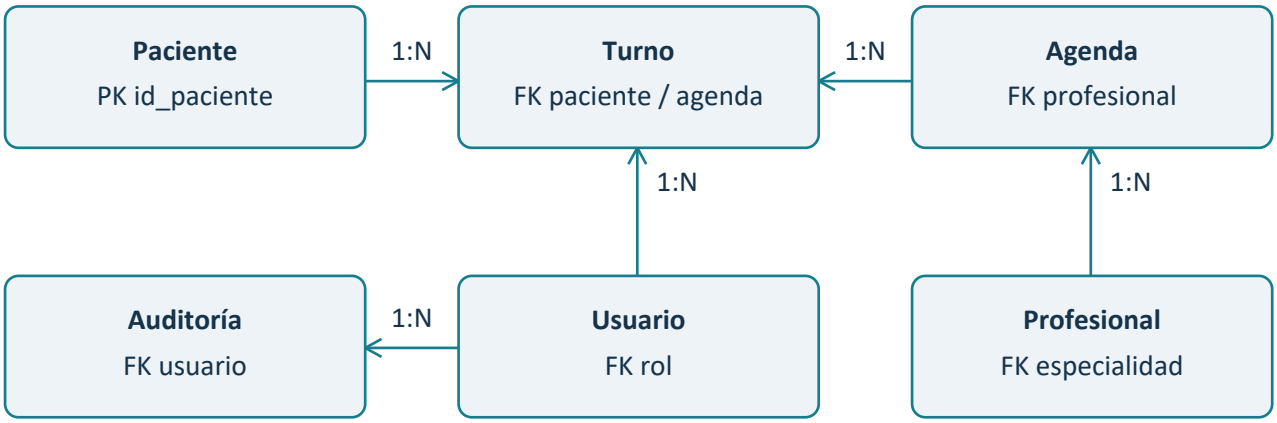
Interfaces y polimorfismo

La interfaz TurnoRepository satisface la exigencia de utilizar clases abstractas o interfaces. En ejecución se inyecta JdbcTurnoRepository; en las pruebas se inyecta Memoria. El mismo servicio opera con ambos objetos a través del contrato. Se conserva la representación de Turno como record y sus valores tipados con LocalDate, LocalTime y TurnoEstado.

DAO es un patrón de acceso a datos; la arquitectura por capas organiza el sistema completo. No se presenta la arquitectura como si fuera el patrón. El costo de esta separación es mantener un contrato y clases adicionales, a cambio de poder modificar la persistencia y probar el servicio de forma aislada.

4. Modelo relacional y persistencia MySQL

El paquete repository contiene las clases de persistencia. El esquema se entrega en sql/schema.sql y utiliza InnoDB, claves primarias y foráneas, restricciones CHECK e índices. Se crea sigtas_ap4 como base separada para conservar sigtas. El script no elimina bases existentes y debe ejecutarse una sola vez.



Turno y auditoría se escriben en una misma transacción.

Figura 2. Vista parcial del modelo relacional, centrada en la reserva. Una agenda corresponde a un profesional y tiene muchos turnos; cada turno pertenece a un paciente. Usuario identifica al actor de creación o modificación. Auditoría referencia al usuario y registra entidad e identificador; entidad_id no es una clave foránea a turno.

El esquema completo conserva rol, usuario, especialidad, profesional, paciente, agenda, turno, registro_atencion y auditoria. Los catálogos y registro_atencion quedan fuera de las operaciones del menú. Los datos de prueba son ficticios y los registros históricos se mantienen para consultar el funcionamiento inicial.

Conexión, consulta y actualización

DatabaseConnection.getConnection obtiene SIGTAS_DB_URL, SIGTAS_DB_USER y SIGTAS_DB_PASSWORD. DriverManager abre una conexión JDBC. Cada operación cierra Connection, PreparedStatement y ResultSet mediante try-with-resources; las credenciales no se escriben en el código.

agendaDiaria filtra por fecha y ordena por hora_inicio. Cada fila se convierte en un Turno mediante map y se agrega a una lista. Main.mostrarAgenda presenta ID, agenda, paciente, horario y estado. guardar inserta el turno y devuelve su clave generada; cancelar actualiza estado, motivo_cancelacion y modificado_por, sin borrar la fila.

Reglas de integridad

Antes de reservar se comprueba que el paciente exista y esté activo, que profesional y agenda estén activos y que la fecha coincida con el día configurado (1=lunes, 7=domingo). El horario debe estar dentro de la agenda, durar el número de minutos definido y comenzar en una posición válida de su grilla. Las claves foráneas rechazan identificadores inexistentes.

5. Transacciones y manejo de excepciones

El alta se ejecuta con autoCommit desactivado y aislamiento READ_COMMITTED. Primero se identifica y bloquea la fila del profesional con SELECT ... FOR UPDATE; después se bloquea y valida la agenda. Esto coordina las reservas del mismo profesional, incluso si pertenecen a agendas diferentes. El bloqueo se mantiene hasta finalizar la transacción (MySQL, s. f.).

Detección de superposición

t.hora_inicio < nuevoFin AND t.hora_fin > nuevoInicio

La condición detecta cruces parciales o totales entre intervalos del mismo profesional y fecha en estados ASIGNADO, PRESENTE o ATENDIDO. Se permiten turnos contiguos: uno que termina a las 09:30 no se superpone con otro que empieza a las 09:30. Los cancelados dejan de bloquear la franja.

Unidad transaccional

cn.setAutoCommit(false);

// Validar, insertar turno y registrar auditoría.

cn.commit();

Si una validación o sentencia falla antes de confirmar, se invoca rollback. La cancelación también agrupa UPDATE y auditoría en una transacción. Solo se actualiza un turno ASIGNADO; si la cantidad de filas afectadas no es uno, la operación se rechaza. Así se evita repetir una cancelación o cancelar un turno ya atendido. JDBC permite confirmar o revertir el conjunto de sentencias (Oracle, s. f.-b).

Excepción	Tratamiento implementado
IllegalArgumentException	Comunica datos inválidos o reglas de negocio incumplidas. No continúa el alta.
DateTimeParseException	Indica el formato esperado para fecha y hora.
SQLException	Se propaga desde persistencia. La consola informa que no se confirmó la operación y muestra SQLState.
IOException	Informa problemas al guardar o recuperar el archivo CSV.

El repositorio captura SQLException y RuntimeException dentro de la transacción, intenta rollback y vuelve a propagar la excepción. Si la reversión también falla, agrega ese error como excepción suprimida para no perder la causa original. Los recursos se cierran al salir del bloque.

Límite del control: el protocolo protege las reservas que pasan por este DAO. Las escrituras externas deben respetarlo. No se implementan reintentos automáticos por interbloqueo. Una desconexión durante COMMIT puede dejar el resultado incierto; se debe consultar agenda y auditoría antes de reintentar.

6. Arreglos, ArrayList y archivos

ArrayList para una cantidad variable de turnos

List<Turno> result = new ArrayList<>();

while (rs.next()) result.add(map(rs));

En JdbcTurnoRepository.agendaDiaria, la cantidad de registros se conoce al ejecutar la consulta. ArrayList permite incorporar cada resultado sin fijar previamente un tamaño. Se devuelve como List para que el consumidor dependa de una abstracción. ArrayList es una implementación de lista basada en un arreglo redimensionable (Oracle, s. f.-c).

Arreglo para un resumen de categorías fijas

int[] cantidades = new int[TurnoEstado.values().length];

for (Turno turno : turnos)

  cantidades[turno.estado().ordinal()]++;

ResumenAgenda.contar declara e inicializa un arreglo de enteros, cuyos elementos comienzan en cero. Recorre los turnos y aumenta el contador correspondiente. Main obtiene además TurnoEstado[] estados y usa un for con índice para mostrar cada estado y su cantidad.

Índice	Estado	Ejemplo
0	ASIGNADO	2
1	PRESENTE	0
2	ATENDIDO	0
3	CANCELADO	1

El ejemplo se verifica en arregloCuentaListaVariable. La lista conserva el detalle de tres turnos; el arreglo produce [2, 0, 0, 1]. Para una lista vacía devuelve cuatro ceros. El orden del arreglo se deriva del mismo enum tanto al contar como al mostrar; no se persisten índices ordinales en MySQL.

Persistencia complementaria en archivos

ArchivoAgenda.guardar recorre la lista, genera un CSV con encabezado y lo escribe en UTF-8 mediante Files.writeString. Incluye ID, paciente, agenda, fecha, inicio, fin y estado. Los campos exportados son valores numéricos, temporales y enum, sin texto libre que requiera comillas CSV.

Las opciones 4 y 5 permiten guardar y recuperar exportaciones/agenda-AAAA-MM-DD.csv. leer usa Files.readString y presenta el contenido en consola. Exportar de nuevo la misma fecha reemplaza el reporte anterior. La recuperación permite consultar el archivo; no importa turnos a MySQL ni reconstruye toda la base.

Esta función opcional permite practicar archivos y ofrece un reporte transportable. MySQL sigue siendo la fuente operativa. La prueba archivoGuardaYRecupera comprueba escritura, lectura y error por archivo inexistente.

7. Verificación y trazabilidad

Resultado ejecutado: compilación y empaquetado exitosos con JDK 17.0.20.1 y Maven 3.9.11. PrototipoTest ejecutó ocho pruebas: cero fallos, cero errores y cero omitidas. Además, MySqlIntegrationTest aprobó dieciséis pruebas contra MySQL 8.4.13 el 07/11/2026; se verificó una sesión completa de la consola. La evidencia se adjunta en docs/RESULTADO-UNITARIAS.txt.

Prueba unitaria	Resultado
Alta válida delega y conserva estado ASIGNADO	Aprobada
Identificador inválido y fecha pasada (2 pruebas)	Aprobadas
Intervalo invertido y fecha nula (2 pruebas)	Aprobadas
Motivo vacío o mayor a 255 caracteres	Aprobada
ArrayList, resumen por estado y lista vacía	Aprobada
Guardar/recuperar CSV y archivo inexistente	Aprobada

Integración ejecutada: se aprobaron dieciséis escenarios en una base aislada: consultas, altas, cancelaciones, claves foráneas, horario, archivos, concurrencia y rollback ante un fallo provocado de auditoría. Se obtuvo BUILD SUCCESS: 24 pruebas, cero fallos, cero errores y cero omitidas. La evidencia está en docs/RESULTADO-INTEGRACION.txt y docs/evidencia/.

Requisito	Evidencia en el proyecto
Patrón e interfaz	TurnoRepository, JdbcTurnoRepository y constructor de TurnoService.
MySQL e interfaz	DatabaseConnection, agendaDiaria, guardar, cancelar y menú Main. Validación real aprobada con MySQL 8.4.13.
Excepciones	Capturas específicas en Main; reversión en JdbcTurnoRepository.
Arreglos y ArrayList	ResumenAgenda.contar y agendaDiaria; prueba automatizada aprobada.
Archivos	ArchivoAgenda y opciones 4/5; prueba automatizada aprobada.
Video y participación	Video final con voz original y salida real de consola. La participación se acredita personalmente.

Criterios comprobados: el alta conserva turno y auditoría; una reserva superpuesta se rechaza; cancelar mantiene el historial; dos solicitudes concurrentes producen una sola reserva. También se probó que un fallo de auditoría revierta el turno y que dos intervalos contiguos sean válidos. La sesión de consola se contrastó con consultas directas en MySQL.

8. Instalación, entrega y conclusiones

1. Instalar JDK 17 o superior, Maven y MySQL 8.0.16 o superior con InnoDB. Importar pom.xml como proyecto Maven en una IDE Java y seleccionar el JDK. La validación registrada se realizó desde terminal. Se entrega configuración de ejecución para VS Code y recomendaciones de extensiones Java.

2. Ejecutar sql/schema.sql y luego sql/data.sql en MySQL Workbench o en el cliente MySQL. La base creada se denomina sigtas_ap4. Configurar las tres variables de entorno de conexión descritas en README.md, sin incorporarlas al repositorio.

3. Ejecutar **mvn clean package** para compilar y probar. Iniciar con **mvn exec:java** o ejecutar ar.edu.sigtas.ui.Main desde la IDE. El JAR generado no empaqueta las dependencias; el inicio con Maven resuelve el controlador JDBC.

4. Para demostrar el alta, usar paciente 1, agenda 1, usuario 2 y un lunes futuro de 09:00 a 09:30. Consultar la fecha, repetir la reserva para observar el rechazo, cancelar el ID creado, consultar nuevamente y exportar/recuperar el CSV. Los datos históricos de septiembre de 2026 sirven para consultar, no para reservar en el pasado.

Repositorio y presentación

Repositorio del proyecto: https://github.com/PabloRuizS2/SIGTAS. Se verificó su existencia y se tomó el código público como base de adaptación. La versión final se encuentra en la carpeta AP4 del repositorio, conservando las entregas anteriores. Enlace directo: github.com/PabloRuizS2/SIGTAS/tree/main/AP4. El código está organizado en src/main/java, src/test/java, sql y docs.

El video final dura 3 minutos y 9 segundos, con la voz original de Pablo Ruiz, subtítulos y pantallas sincronizadas. La demostración reproduce la salida real de Main contra MySQL, registrada el 07/11/2026: alta, consulta, rechazo, cancelación y recuperación del CSV. La participación en foros, encuentros y espacios comunes requiere evidencia personal y no se acredita mediante este desarrollo.

Limitaciones y evolución

El prototipo no incluye autenticación real, interfaz gráfica, facturación, altas de catálogos desde consola ni gestión clínica. Conserva los estados de la versión anterior, pero el menú solo registra ASIGNADO y permite pasar a CANCELADO. No controla coincidencias horarias del mismo paciente con profesionales diferentes. El bloqueo por profesional puede reducir el paralelismo y deberá medirse antes de ampliar el sistema.

Se mantiene la organización ficticia utilizada en el TP3 para preservar la continuidad. No se dispone de observaciones docentes adicionales ni de los documentos completos AP1 y AP2; no se afirma una revisión de material no adjunto.

Conclusión

La entrega integra un prototipo Java con contrato DAO, consultas y actualizaciones MySQL, transacciones, excepciones, arreglos, ArrayList y archivos. La compilación, las ocho pruebas unitarias y las dieciséis pruebas MySQL aportan evidencia concreta. La sesión real de consola y el video permiten comunicar el alcance del prototipo. El código y sus archivos asociados se organizan en la carpeta AP4 del repositorio.

9. Referencias

Ruiz, P. (2026). *Sistema de Gestión de Turnos y Atención del Centro Médico Salud Integral (SIGTAS): TP3* [Trabajo académico no publicado]. Universidad Siglo 21.

Ruiz, P. (s. f.). *SIGTAS* [Código fuente]. GitHub. Recuperado el 29 de octubre de 2026, de https://github.com/PabloRuizS2/SIGTAS. Base consultada: commit 17453aaa983e2b59d6d00b793f7edf92cba04259.

Universidad Siglo 21. (2026). *Seminario de Práctica de Informática: actividad 4, consigna abierta y rúbrica* [Material de aula virtual facilitado por el estudiante].

Oracle. (s. f.-a). *Core J2EE Patterns: Data Access Object.* https://www.oracle.com/java/technologies/dataaccessobject.html

Oracle. (s. f.-b). *Using transactions.* The Java Tutorials. https://docs.oracle.com/javase/tutorial/jdbc/basics/transactions.html

Oracle. (s. f.-c). *ArrayList.* Java Platform, Standard Edition 8 API Specification. https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html

MySQL. (s. f.). *Locking reads.* MySQL 8.4 Reference Manual. https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html

Material complementario de la entrega

El proyecto incluye README.md, los scripts SQL, el código Java, las pruebas unitarias, la matriz de trazabilidad, los casos de integración y el guion de video. Las figuras 1 y 2 son de elaboración propia para esta implementación. La documentación Java 8 se utiliza como referencia de las APIs estables citadas; el proyecto se compila con Java 17.